第6回　Python II：CSV、pandas、plot

CSVと観測単位、pandasで読む、型・日付・欠損、図の1点をCSVの行まで戻る

2026年10月20日（火）3限　金融論

この回で扱うこと

- CSVと観測単位
- pandasで読む、列と行を選ぶ
- 型・日付・欠損・並べ替え
- matplotlibで図を描き、図の1点をCSVの行まで戻る
- 同じ処理をCodexに頼む書き方

この回のcodeは、自分で打つ場合と、Codexに頼んで書かせる場合の両方を載せています。Codexに頼むときの指示文は、各節の `text` の枠にあります。どちらの場合も、出てきた表や図が正しいかを確かめるのは自分です。何を確かめるかは、codeを読める人のほうが早く決められます。

実習の準備

この回は、Documents（書類）に自分の作業用フォルダ `lecture06` を作り、空のnotebookにcodeを書き足して進めます。

`lecture06` の中に `data` フォルダを作ります。配布資料の `workbook/data/` にある `prices.csv` と `prices_short.csv` を、その中にコピーします。

VS Codeの「ファイル」→「フォルダーを開く」で、Documentsの `lecture06` を開きます。左のファイル一覧で「新しいファイル」から `lecture06.ipynb` を作ると、空のnotebookが開きます。右上の `Select Kernel` から、HW01で使った `finance-env` を選びます。

準備ができると、ファイルは次の配置になります。

```
Documents/
└── lecture06/
    ├── lecture06.ipynb
    └── data/
        ├── prices.csv
        └── prices_short.csv
```

本文のPython codeをcode cellに入力し、Shift+Enterで実行すると、結果がcellの下に出ます。`+ Code` でcellを追加できます。⌘S（WindowsはCtrl+S）で保存すると、codeと結果が `lecture06.ipynb` に残ります。

この配置なら、本文の `data/prices.csv` や `data/prices_short.csv` は、notebookと同じフォルダにある `data` の中のfileを指します。

配布資料の `workbook/workbook.ipynb` には、同じ処理と追加の練習が入っています。「やってみる」の答え合わせは、Codexに頼めます。`workbook/workbook.py` は、同じ処理をscriptとして上から実行できる形にしたものです。

練習用の株価CSV

この回は、自動車2社と小売1社の架空の株価を使います。

銘柄コード	名前	入れてあること
`01110`	Auto A	分割や欠損はありません
`02220`	Auto B	2025年4月1日に1株を2株に分割
`03330`	Retail C	2024年8月5日は売買が成立せず、価格が欠損、出来高が0

銘柄コードは、0から始まる5文字にしてあります。実在の銘柄と重ならず、読み方によって先頭の0が消えることを確かめられます。

`prices.csv` は、2024年1月4日から2025年12月30日までの、東証の営業日488日分です。日付は実際の暦で、価格は架空です。3社で1,464行あります。`prices_short.csv` はそのうち2024年8月1日・2日・5日の終値と出来高だけの表で、並び順と数の書き方をわざと崩してあります。

列の名前はJ-Quants APIの株価と同じです。この回に書いたcodeは、第7回に取得する実在企業の表にもそのまま使えます。

CSVとJSON、観測単位

CSVはtext file

CSVは、値を `,` で区切って並べたtext fileです。

```
Date,Code,C,Vo
2024-08-05,01110,"2,467",27465800
2024-08-05,03330,,0
```

1行目が見出しで、列の名前が並びます。2行目から下が値です。値の中に `,` があるときは、`"2,467"` のように引用符で囲みます。値がないところは、`,,` のように何も書かれていません。

VS Codeで開くと、書かれている文字がそのまま見えます。Excelで開くと、Excelが値を解釈して表示します。`01110` は数の1110に、`2024-08-05` は日付に、`"2,467"` は数の2467になります。Excelで上書き保存すると、解釈された形でfileが書き換わります。

JSONとは

JSON（JavaScript Object Notation）は、データを「名前と値の組」や「値の並び」で表すtext形式です。fileに保存したり、APIでデータを受け渡したりするときに使います。名前にJavaScriptとありますが、Pythonなど多くの言語で読み書きできます。

上のCSVと同じ2行をJSONで書くと、次のようになります。

```
{"data": [
  {"Date": "2024-08-05", "Code": "01110", "C": 2467, "Vo": 27465800},
  {"Date": "2024-08-05", "Code": "03330", "C": null, "Vo": 0}
]}
```

1行が `{ }` で囲んだ「列の名前と値の組」です。それが `[ ]` の中に並び、`"data"` という名前の下に入っています。CSVは表だけを書く形式ですが、JSONはこのように入れ子にできるので、行の並びと、それ以外の情報を同じfileに入れられます。第7回で使うAPIは、この形で返答します。

CSVと比べると、値の種類の扱いが違います。

- `"01110"` は引用符が付いているので文字列です。先頭の0は消えません。CSVで `dtype={"Code": str}` が要るのは、CSVにこの区別がないからです。
- `2467` は引用符がないので数です。CSVの `"2,467"` のように、桁区切りのために引用符で囲む必要がありません。
- 値がないところは `null` と書きます。CSVの空欄と同じく、pandasでは `NaN` になります。
- `Date` は文字列です。JSONにも日付の型はないので、`pd.to_datetime` はどちらの形式でも要ります。

1行が何を表すか

`prices.csv` の1行は、1つの銘柄の1日です。3社×488日で1,464行になります。

Date	Code	C
2024-01-04	01110	2600.0
2024-01-05	01110	2670.0
…	…	…
2024-01-04	02220	3000.0

1行を1日にして、銘柄ごとに列を並べる形もあります。今回の表は、銘柄と日付の組が1行で、列は価格や出来高の種類です。J-Quantsから取得する表も、この形です。

pandasとは

pandas（パンダス）は、Pythonで表のデータを読み込み、整理し、集計するためのlibraryです。libraryは、よく使う処理をまとめたcodeの集まりです。Python本体にpandasを追加すると、CSVやExcelのfileを読み、行や列を選び、平均や合計を求める機能を使えます。

たとえば株価の表なら、ある銘柄の行だけを取り出す、日付順に並べる、前日からの変化率を計算する、といった処理ができます。数値だけでなく、銘柄名のような文字列、日付、欠けた値も扱えます。

Excelではcellを選んだり数式を入力したりして表を操作します。pandasでは、その操作をPythonのcodeで書きます。codeを保存しておけば、新しいデータにも同じ処理をまとめて実行できます。この回では、株価CSVをpandasで整理し、その結果をmatplotlibに渡して図を描きます。

fileとDataFrame

DataFrameは、pandasが表を扱うためのデータの形です。行と列があり、列ごとに名前と型（整数、小数、文字列、日付）を持ちます。Excelのシートに似ていますが、列全体をまとめて計算したり、条件に合う行だけを取り出したりする操作が、1行のcodeでできます。第5回のlistが「値の並び」だとすれば、DataFrameは「名前の付いた列の並び」です。

DataFrameの作り方は2つあります。1つは、この回のようにCSV fileを `pd.read_csv` で読む方法です。もう1つは、`pd.DataFrame({"Code": ["01110", "02220"], "C": [2600.0, 3000.0]})` のように、列名をkey、値のlistを値にした辞書から作る方法です。第8回の会計の表は、後者の形で小さな表を自分で作ります。

CSV fileは、diskに保存された文字の並びです。pandasで読むと、kernelのメモリの中にDataFrameができます。DataFrameの列を書き換えても、CSV fileは変わりません。書き換えた表は、`to_csv` で書き出すとfileに残ります。

---

pandasで読む

import と read_csv

HW01で `finance-env` に入れたpandasを、`import` で読み込みます。`pd` は慣例の略です。

```
import pandas as pd

short = pd.read_csv("data/prices_short.csv")
```

`import pandas as pd` で、pandasを `pd` という短い名前で呼べるようにします。`pd.read_csv` はCSVを読み、DataFrameを返す関数です。

`short.head()` は先頭の5行、`short.shape` は行数と列数、`short.dtypes` は列ごとの型を返します。何も指定せずに `prices_short.csv` を読むと、`shape` は `(9, 4)` です。`Code` は `int64`（整数）で、`01110` が `1110` になっています。`C` は `"2,467"` に `,` が入っているので、`object`（文字列）です。

Codexに頼むなら、作業用の `lecture06` フォルダを開いた状態で次のように書きます。

```
data/prices_short.csvをpandasで読んで、先頭5行、行数と列数、列ごとの型を表示して。
```

返ってきた型の一覧で、`Code` が整数になっていないか、`C` が文字列のままでないかを見ます。Codexは指示どおりに読むだけなので、この2つは自分で気づく必要があります。

列と行を選ぶ

```
short["C"]
short[["Date", "Code", "C"]]
short[short["Code"] == "01110"]
```

列の名前を `[ ]` に入れると、その列が出ます。複数の列は、名前のlistを入れます。

行を選ぶときは、`short["Code"] == "01110"` で行ごとの `True`・`False` を作り、それを `short[ ]` に入れます。`True` の行だけが残ります。`Code` を整数として読んでいると、`"01110"` と一致する行がなく、空の表が返ります。errorは出ないので、行数を見て初めて分かります。

listとDataFrame、関数とmethod

listとDataFrameの列では、同じ `* 2` でも結果が違います。

```
[27465800, 10278200, 0] * 2    # listが2回繰り返される
short["Vo"] * 2                # 列の値が行ごとに2倍になる
```

`len(short)` は、関数 `len` に表を渡す書き方です。`short.head()` は、表が持っているmethod `head` を呼ぶ書き方です。`short.shape` のように `( )` を付けないものは、表が持っている値で、`short.shape()` と書くと `TypeError` になります。

`short` はPythonの変数の名前で、`"C"` は表の中の列の名前です。列の名前は文字列なので、引用符で囲みます。

型・日付・欠損・並べ替え

銘柄コードは文字列

```
short = pd.read_csv("data/prices_short.csv", dtype={"Code": str})
```

`dtype={"Code": str}` を付けて読むと、先頭の0が残ります。 銘柄コードは、足したり平均したりする数ではなく、銘柄を区別する記号です。第10回で複数の表を銘柄コードで結合するときも、片方の `Code` が文字列、もう片方が整数だと、結合の段階でerrorになります。

数に見える文字列

`"2,467"` は数に見えますが、文字列です。文字列の列に2を掛けると、`2,4672,467` のように文字がつながります。`,` を消してから数に変換します。

```
short["C"] = pd.to_numeric(short["C"].str.replace(",", ""))
```

`,` を残したまま `pd.to_numeric` を使うと、`ValueError` で止まります。空欄だった値は `NaN` になります。

型の直しは、Codexにまとめて頼めます。

```
data/prices_short.csvを、Codeは文字列、Dateは日付型、Cは「,」を外して数値として読んで。
読んだ後の型の一覧と、Codeの値の一覧を表示して。
```

`Code` に `01110` のように先頭の0が残っているか、`Date` が `datetime64` になっているかを確かめます。

日付

タイムスタンプは、ある時点の日付や時刻を表す値です。例えば `2024-08-05 15:00:00` は、2024年8月5日15時を表します。今回の株価データは1日単位なので、`Date` には観測日だけが入っています。

読んだ直後の `Date` は、`2024-08-05` という文字列です。見た目は日付でも、文字列のままでは期間で絞ったり、前の日との間隔を数えたりできません。`pd.to_datetime` で日付型（`datetime64`）にすると、それができるようになります。読むときに `parse_dates=["Date"]` を付けても、同じ型になります。

書き方が `2024/8/5` のようにそろっていないfileでは、もう1つ問題が起きます。文字列は左から1文字ずつ比べるので、`"2024/10/1"` が `"2024/8/5"` より前に並びます。日付型にすると、日付の順に並びます。

並べ替え

`prices_short.csv` は新しい日付が上にあります。`sort_values(["Code", "Date"])` は、まず銘柄で、同じ銘柄の中では日付で並べます。前の日との変化率は、この並びを前提に計算します。

Codexに頼むなら、次のように書きます。

> shortを、Codeの昇順、同じCodeの中ではDateの古い順に並べ替えて。
> Dateが文字列なら、日付型に変換してから並べ替えて。
> 結果をshortに保存し、Code、Date、C、Voの列を表示して。

表示された表で、同じ銘柄の行がまとまり、その中で日付が古い順に並んでいることを確かめます。

欠損と0

Retail Cの2024年8月5日は、終値が空欄、出来高が0です。売買が成立しなかった日です。

- 終値は、取引がなかったので値がありません。pandasでは `NaN`（欠損）になります。
- 出来高は、売買された株数が0株だったので、0という値があります。

`NaN` は Not a Number の略で、数の列に値がないことを表す印です。

`isna().sum()` で、列ごとの欠損の数が分かります。平均を計算すると、欠損の扱いで結果が変わります。

計算	Retail Cの3日間の終値の平均
`mean()`（欠損を飛ばす）	3,893.00円
`fillna(0).mean()`（欠損を0円にする）	約2,595.33円

欠損を0で埋めると、売買のなかった日の株価を0円とみなしたことになります。

Codexに「欠損を処理して」とだけ頼むと、0で埋めるか、行を落とすか、Codexが決めてしまいます。どう扱うかを指示に書きます。

> 列ごとの欠損の数を表示して。終値の欠損は埋めずにそのまま残し、平均は欠損を除いて計算して。

CSVは型を保存しない

`to_csv` で書き出してから `read_csv` で読み直すと、`Code` はまた整数に、`Date` は文字列に戻ります。CSVには文字しか残らないからです。`dtype` と `parse_dates` を付けて読めば、同じ型に戻ります。

2年分の表を読む

```
prices = pd.read_csv("data/prices.csv", dtype={"Code": str}, parse_dates=["Date"])
```

1,464行14列です。`prices["Code"].value_counts()` で、3社とも488行あることが分かります。列の意味と単位は次のとおりで、第7回のJ-Quantsの表と同じです。

列	内容	単位
`Date`、`Code`	取引日、銘柄コード	
`O`、`H`、`L`、`C`	始値、高値、安値、終値	円
`Vo`	出来高	株
`Va`	売買代金	円
`AdjFactor`	調整係数。分割の日だけ1以外	
`AdjO`、`AdjH`、`AdjL`、`AdjC`、`AdjVo`	分割などを調整した価格と出来高	円、株

期間で絞るときは、2つの条件を `&` でつなぎ、それぞれを `( )` で囲みます。

```
aug = prices[(prices["Date"] >= "2024-08-01") & (prices["Date"] <= "2024-08-31")]
```

2024年8月は21営業日なので、3社で63行です。

新しい列は、列どうしを計算して代入すると作れます。銘柄コードに名前を付けるときは、辞書（dict）と `map` を使います。

```
prices["HLmean"] = (prices["H"] + prices["L"]) / 2

names = {"01110": "Auto A", "02220": "Auto B", "03330": "Retail C"}
prices["Name"] = prices["Code"].map(names)
```

ここまでをCodexに頼むと、次のようになります。

```
data/prices.csvを、Codeは文字列、Dateは日付型で読んで。
2024年8月の行だけを取り出して行数を表示して。
高値と安値の平均の列HLmeanと、銘柄コードに対応する名前の列Name（01110はAuto A、02220はAuto B、03330はRetail C）を足して。
```

8月の行数が63行になっているか、`Name` に `NaN` が混じっていないかを見ます。

図を描く

matplotlibの基本

```
import matplotlib.pyplot as plt

auto_a = prices[prices["Code"] == "01110"]
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(auto_a["Date"], auto_a["C"])
ax.set_title("Auto A: closing price, 2024-01-04 to 2025-12-30")
ax.set_xlabel("Date")
ax.set_ylabel("Yen per share")
fig.text(0.99, 0.01, "Source: Fictional data", ha="right", fontsize=8)
plt.show()
```

`fig` は図全体、`ax` は線を描く座標軸です。`ax.plot(横軸, 縦軸)` で折れ線になります。タイトルに銘柄と期間、縦軸に単位、右下に出典を書いておくと、図を別の資料に貼っても何の数値かが分かります。

図はCodexに頼むことが多くなります。タイトル、軸の名前、出典、保存先まで指示に入れると、あとで直す手間が減ります。

```
pricesからAuto A（Code 01110）の終値Cの折れ線を描いて。
タイトルは銘柄名と期間、縦軸は円、右下に「Source: Fictional data」と入れて。
matplotlibで描き、output/auto_a_close.pngに保存して、notebookにも表示して。
```

横軸が日付順に並んでいるか、縦軸の範囲が0から始まっていないことで変化が誇張されて見えないか、線が1本だけかを見ます。

株式分割と調整後の価格

Auto Bの終値は、2025年3月31日の2,227円から、4月1日に1,117円になります。1株が2株に分かれたからです。この日の `AdjFactor` は0.5です。

`AdjC` は、分割前の価格に0.5を掛け、分割後の株数の基準にそろえた値です。3月31日は1,113.5円で、4月1日の1,117円とつながります。2年間を `C` で描くと、Auto Bは3,000円から1,440円へ、半分以下になったように見えます。`AdjC` では1,500円から1,440円です。

分割をまたいで株価を比べるときは、調整後の列を使います。

3社を比べる

`for` で銘柄を1つずつ選び、同じ `ax` に線を重ねます。株価の水準が違う銘柄どうしは、最初の日の値で割って100を掛けると、値動きの割合で比べられます。2024年1月4日を100とすると、2025年12月30日はAuto AとRetail Cが113.8、Auto Bが96.0です。

```
3社の調整済み終値AdjCを、2024年1月4日を100として1つの図に重ねて描いて。凡例は銘柄名にして。
最後の日の3社の値も表示して。
```

Codexが `C` を使っていないか、表示された最後の値が113.8、113.8、96.0になっているかを確かめます。

変化率の分布

前の営業日からの変化率は、`shift(1)` で1行上の値を取って計算します。

```
change = (auto_a["AdjC"] - auto_a["AdjC"].shift(1)) / auto_a["AdjC"].shift(1) * 100
```

Auto Aでは487日分の変化率ができ、最小は約-4.85%、最大は約5.15%です。Auto Bを調整前の `C` で計算すると、分割の日に-49.84%という値が1つだけ現れます。`AdjC` で計算すると、この日は0.31%です。

```
銘柄ごとに、前の営業日からの変化率（%）をAdjCから計算した列Changeを足して。
銘柄ごとの最小と最大、その日付を表示して。
```

Codexは銘柄をまたいで `shift` してしまうことがあります。各銘柄の最初の日の `Change` が `NaN` になっていれば、銘柄ごとに計算できています。

図の1点をCSVの行まで戻る

`idxmax()` は、最大値のある行の番号を返します。Auto Aの調整後終値の最大は、行番号483の2025年12月24日、3,119.0円です。

`prices` は `prices.csv` を並べ替えずに読んだので、行番号0がCSVの2行目（見出しの次）にあたります。行番号483はCSVの485行目で、VS Codeで開くと同じ3119.0が並んでいます。図の赤い点、DataFrameの行、CSV fileの行が、同じ1つの値を指していることが確かめられます。

どの列で最大を探すかによって、指す行は変わります。Auto Bで調整前の `C` の最大を探すと、分割前の2024年4月5日、3,210円です。`AdjC` の最大は2025年11月27日の1,689円です。

```
Auto AのAdjCが最大の行を、行番号、日付、値とともに表示して。
その値がdata/prices.csvの何行目にあるかも示して。
```

VS Codeでその行を開き、同じ値が並んでいることを自分の目で確かめます。図とDataFrameとCSVが同じ1点を指すことは、Codexの答えではなく、fileを開いて確かめます。

notebookの最後では、Restart と Run All で上から全部を実行し直します。CSVを読む、型を直す、並べ替える、図を描く、の順に実行され、同じ表と図がもう一度できます。

この回の要点

- CSVは文字だけのfileで、型を保存しません。銘柄コードは `dtype={"Code": str}`、日付は `parse_dates` を付けて読みます。
- CSVもJSONも文字だけのfileですが、JSONは文字列と数を引用符で区別し、入れ子にできます。日付の型はどちらにもありません。
- 価格の欠損（`NaN`）と出来高の0は、意味が違います。分割をまたいで比べるときは `AdjC` を使います。
- 図の1点は、`idxmax` などで行番号を出し、CSVの行まで戻ると確かめられます。